

Omega: A Programming Language Built for Verified Programs

M. Drake Stapleton

October 2026

Abstract

We propose Omega, a planned systems programming language that combines the memory safety of Rust with the simplicity of Go, and adds two properties neither language has: it is semantics-first, and verification is a first-class language feature. In Omega, the primary object is meaning, not text. Syntax is a disposable skin over a canonical, content-addressed semantic identity: two surface texts that elaborate into the identical normalized semantic graph share one semantic identity (SEMANTIC_ID). The module system is the crumb store. A *crumb* is a verified program together with its verification receipt, and programs may only be built from verified crumbs. The doctrine that the verified program is the universal artifact of meaning is therefore not a policy document. It is the grammar of the language. In this document we state the motivation, the design principles, the intended trade-offs, and the open questions, and then we answer the objections we expect. It is a design proposal, not a specification. Nothing here is final until we have built, measured, and ratified it.

1 Motivation

We propose a programming language built for verified programs.

Most programming languages were designed for a world in which humans write all the code. That world is ending. In our work, the large majority of code is already written by AI agents, reviewed by AI agents, and tested by AI agents. Languages optimized for human comfort are optimizing for the wrong writer.

Rust is the closest existing answer to safe systems programming, but its compiler is a harsh teacher: the learning curve is steep, builds are slow, and the strictness that makes it safe also makes it unfriendly. Go chose the opposite trade: simple, fast to build, friendly. It keeps memory safety apart from data races, and pays for simplicity with a garbage collector and less fine-grained control over layout and timing.

Omega starts from a different question, and we ask it directly: what would a language look like if we designed it for verifiability first, with the knowledge that its primary authors will be machines? A strict compiler is painful for a human. For an agent, a strict compiler is a tireless reviewer that never sleeps, never gets tired, and catches the bug before it ships. The pain of strictness is almost entirely human pain. We design Omega to spend that budget where it buys the most: correctness you can check, not comfort you can feel.

The name is deliberate. In our architecture, Omega is the umbrella system that defines, synthesizes, and coordinates verification. We give the language the same name because the language is one

interface of that system. Verification itself is performed by an independent verifier, never by the generator: the language does not grade its own work.

2 Semantics first

In every mainstream language, the primary object is text. The compiler reads characters, and meaning is something the text is hoped to carry. Omega inverts this: the primary object is meaning, and text is a disposable skin.

This is not a metaphor. It is the founding position of the Omega engine, established in Milestone 4 (OMEGA_SEMANTICS): Omega defines what computation means without committing that meaning to any particular machine representation. The governing axiom is that meaning is permanent and representation is disposable. The precise claim is this: SEMANTIC_ID is the hash of the canonical semantic IR produced by elaboration, and the hash input includes the semantic schema version and the normalization rules version: $\text{SEMANTIC_ID} = \text{H}(\text{schema_version} \parallel \text{normalization_version} \parallel \text{canonical_graph_bytes})$. A stable byte-level graph cannot silently acquire a new interpretation as the language evolves, because any change to the schema or the normalization rules changes the identity. Two surface texts that elaborate into the identical normalized semantic graph under the same schema and normalization versions share one identity. Programs that elaborate into different graphs may later be proven equivalent, but that proof is a separate artifact: source elaborates to canonical semantic IR to SEMANTIC_ID, and separately, SEMANTIC_ID_A is connected to SEMANTIC_ID_B by an equivalence proof. We do not claim that arbitrary programs can be decided equivalent; general behavioral equivalence is not computable, and Omega does not pretend otherwise. What is permanent is the semantic object; what is disposable is its realization. An optimizer does not change the semantic object. It realizes the same object into a different target representation, and carries a refinement or equivalence receipt connecting the realization to the source semantics.

For the language, we draw four consequences that shape everything below.

First, syntax is not sacred. The concrete text a programmer writes is one of many possible skins over a meaning. We leave the language free to offer multiple surfaces, or to change its surface over time, because the surface was never the program. What the programmer writes is parsed into meaning; what is stored, checked, imported, and verified is meaning.

Second, equality is semantic, over the canonical graph. Two crumbs whose surface texts elaborate into the identical normalized semantic graph share one SEMANTIC_ID: they are the same semantic object, regardless of how they were written. They are not necessarily the same crumb: a crumb is a claim plus its receipts, and different specifications asserted of the same graph give different CLAIM_IDs and therefore different crumbs. We deduplicate semantic objects; we never collapse distinct claims or distinct evidence. This makes the crumb store deduplicating by construction: the library cannot fill up with ten spellings of one idea. Refactoring or optimization that changes the semantic graph is not identity-preserving; it produces a new semantic object, connected to the old one by an equivalence receipt where one is claimed.

Third, verification checks meaning, not text. A receipt does not say “this text passed the checks.” It says “this meaning was verified against this specification.” That is why the crumb module system works at all: when you import a crumb, you import a meaning plus the evidence for it, not a file. Files can lie about what they contain. Canonical encoding and hashing do not prevent that: hashes

bind encoded content; they do not establish semantic correctness. A hash proves the content is the content that was hashed, nothing more. What hashing gives us is binding: a meaning changed after hashing no longer matches its hash, so silent substitution is caught.

Fourth, the language and the engine are one idea. The Omega engine’s semantic substrate is the ground the language stands on. The compiler does not translate text to machine code in the traditional sense; it resolves meaning and realizes it on the target. The language is the notation; the semantic substrate is the substance.

In short: most languages ask “what does this text say?” Omega asks “what does this mean?” and treats everything else as packaging.

3 Design principles

1. **The verified program is the unit of meaning.** Claims are admitted through verification, not prose. Semantics defines behavior; specifications state properties; receipts support those properties under declared assumptions. The language manipulates meanings, not text; syntax is a disposable skin over the semantic identity. This is the Crumb Doctrine, and we make it structural in Omega, not aspirational.
2. **Design for the writer you have.** We assume the language’s code is mostly written by agents and mostly read by verifiers. We optimize error messages, tooling, and language rules for machine authors first. Human readability is a requirement, not the goal.
3. **Safety is not optional.** We provide no mode in which the safety guarantees are silently dropped. Any weakening of safety must be explicit, visible, and auditable. See Section 8 for the resolved answer this raises.
4. **One idea per mechanism.** Each language feature does one thing. Features that overlap are a design failure, not richness.
5. **Name the sacrifice.** Every language gives something up. Go gave up fine-grained control for simplicity. Rust gave up friendliness for safety. We state Omega’s sacrifice explicitly in Section 5. We do not leave it to be discovered later by its users.

4 The crumb module system

In Rust, the ecosystem is organized around crates: reusable packages that programs are built from. In Omega, the corresponding unit is the *crumb*: a verified program together with its verification receipt. We define the module and import system of Omega as the crumb store. There is no separate package registry and no separate proof archive. To import a crumb is to import its receipt.

Concretely, this means:

- A crumb is the smallest unit of reuse. Functions, modules, and libraries are all crumbs: each carries the evidence that it does what it claims.
- We permit a program to depend only on verified crumbs. An unverified program cannot be imported, linked, or built upon. We have the compiler check this, not policy.

- A crumb carries several distinct identifiers, and we do not collapse them. `SEMANTIC_ID` is the hash of the canonical meaning together with the semantic schema version and the normalization rules version (Section 2). `SPEC_ID` is the hash of the property or specification being asserted. `CLAIM_ID` is the hash of the semantic ID concatenated with the specification ID: $H(\text{SEMANTIC_ID} \parallel \text{SPEC_ID})$. `RECEIPT_ID` is the hash of the verifier, the evidence, the environment, the result, and the `CLAIM_ID` the receipt attests to: the `CLAIM_ID` is included in the receipt’s hashed payload, so the receipt is bound to the exact claim it verifies. `SEAL_ID` is the hash of the claim ID, the ordered receipt identifiers, and the store policy ID: $H(\text{CLAIM_ID} \parallel \text{ordered_receipt_ids} \parallel \text{policy_id})$, an immutable snapshot of the evidence accepted at seal time. A crumb is the claim plus at least one qualifying receipt: the logical subject stays stable while evidence accumulates, which matters when independent laboratories add their own attestations years later. The same claim may legitimately acquire several receipts: checked by two independent verifiers, reverified years later, or qualified on two architectures. Those are new receipts, not new claims. A crumb cannot be silently substituted or modified without invalidating everything built on it, because every identifier in the chain is content-addressed.
- Four rules hold across these identifiers, and we state them together so the pages agree. The same canonical graph under the same schema and normalization versions gives the same `SEMANTIC_ID`. A different specification gives a different `CLAIM_ID`, because `SPEC_ID` is part of the hash. A different verification event gives a different `RECEIPT_ID` only when event-specific information (timestamp, verifier version, input digests, environment) is included in the hashed payload: hashing identical payloads yields identical hashes, and that is the point of the scheme. A different set of accepted evidence or a different policy gives a different `SEAL_ID`. We deduplicate semantic objects; we never collapse distinct claims, distinct receipts, or distinct seals.
- We call the growing lineage of crumbs the *Crumbline*: the unbroken record of what the machine actually built and verified.

Our intent is that the crumb doctrine, which today we must enforce on agents by external checks, becomes unavoidable. The easy path and the correct path are the same path, because the language has no other path.

This works because crumbs are semantic objects, not files. A crumb’s identity begins with its `SEMANTIC_ID`, the content address of its canonical meaning. Importing a crumb therefore imports a meaning plus its evidence, each addressed separately. Two surface texts that elaborate into the identical normalized semantic graph resolve to the same `SEMANTIC_ID`; the store cannot accumulate ten spellings of one meaning. See Section 2.

5 Safety model

We give Omega Rust-grade memory safety: programs cannot corrupt memory, and the classic classes of memory bugs (use after free, double free, data races on shared memory, buffer overruns) are ruled out by the language, not by programmer discipline. We pursue Go-like simplicity in expressing that safety: the ownership rules are designed to need fewer annotations and produce clearer errors than Rust’s borrow checker, with more inference and fewer ways to say the same thing.

We state the intended trade up front, per our design principles: **we sacrifice low-level control and build speed.** There is no raw pointer arithmetic in the safe subset, no manual memory layout control, and no custom allocators. Verification takes time, so builds are slower than Go's. In exchange, the programmer, human or agent, gets memory safety plus simplicity plus proof-carrying code. If a use case needs hand-tuned memory layout or the fastest possible build, Omega is the wrong tool, and we have the language say so plainly rather than grow a back door.

6 Concurrency

Go's most beloved feature is its concurrency model: thousands of tiny, cheap tasks that are simple to write and simple to reason about. Rust's concurrency is powerful but widely considered painful. We adopt the Go-like model as the headline direction: lightweight tasks with simple syntax, combined with Rust-grade safety guarantees.

We state plainly that this section is direction, not architecture. Before the concurrency model can carry verification, the following must be defined: the memory model, the effect model, the ownership and aliasing model, cancellation semantics, task lifetime rules, channel semantics, failure propagation semantics, scheduling guarantees, and the treatment of nondeterminism. The decisive question is not whether the syntax looks like Go. It is what semantic object a concurrent Omega program denotes, because verification is meaningless until that object is defined.

7 Verification as a language feature

In most languages, verification happens outside the language: test suites, external analyzers, code review. We make verification a first-class part of the language, but we do not pretend it is one homogeneous act. Verification ranges from milliseconds of type checking to hours of property testing, model checking, differential execution, fuzzing, or hardware-in-the-loop qualification. One word should not cover all of that. We therefore divide the pipeline into four stages, and we name them so their costs stay visible.

`omega check` establishes well-formedness and declared effects: the program is coherent and states the effects it claims. This is the fast, local work: parsing, elaboration, and type checking.

A workspace `omega build` produces an unsealed realization for testing: a thing that runs, carrying no admission into the Crumblin. Building does not verify; it realizes.

`omega verify` produces claim-specific evidence against the specification the claim names. The verification profile says what is checked, how, and at what cost; a receipt for type checking and a receipt for model checking are different artifacts with different weight.

`omega seal` admits the artifact and its qualified dependency closure, minting the crumb: from that point the artifact is addressable by its identifiers and its closure is on the record. Only a sealed crumb may enter the crumb store or satisfy a verified dependency.

An unsealed object may exist locally during development; we do not make the inner edit-build cycle unusable in the name of purity. But we distinguish the workspace graph from the crumb graph. In the workspace, sealed and unsealed objects may depend on each other locally: an unsealed object may depend on unsealed objects during development. The crumb graph is sealed only. Nothing

unsealed may satisfy a sealed dependency, enter the crumb store, or contribute to a sealed top-level program until the entire dependency closure meets the required verification policy. That is how “no receipt, no crumb” survives contact with real development.

What is verified is the meaning, not the text. Two surface texts that elaborate into the identical normalized semantic graph share one `SEMANTIC_ID`, and one verification of that graph serves both. Where the graphs differ, equivalence must be proven as its own artifact; we do not declare it by definition.

We draw three consequences from this. First, the Crumbline grows automatically as a side effect of sealing verified software; no separate step records what was proven. Second, the cost of verification is paid once, at seal time, and amortized over every program that imports the crumb afterward. Third, trust becomes checkable: any consumer of a crumb can re-examine its receipt without trusting its author.

We leave the verification machinery itself and the exact contents of a receipt to the language specification. In this document we establish only that verification is not optional tooling. It is the grammar.

8 The foreign boundary

We have decided. There is no conventional `unsafe` inside Omega.

The argument for an escape hatch was practical: hardware interfaces, foreign code, and extreme performance work may be inexpressible inside the safe subset. The argument against was doctrinal: a language whose meaning is the verified program cannot have a region where programs are unverified but still composable, or the doctrine is a suggestion rather than a grammar. We resolve this by refusing the premise that the hatch must live inside the language.

Low-level components, device interfaces, allocators, drivers, SIMD layouts, DMA, MMIO, and hand-tuned kernels, live outside Omega, in the languages that can express them. Omega imports them only through sealed foreign crumbs: boundary artifacts that carry an explicit ABI, a behavioral contract, platform constraints, an implementation digest, and qualification evidence. The foreign crumb is sealed by the independent verifier like any other crumb, against the contract it claims. Omega never sees the inside; it sees the contract and the receipt, and that is enough to build upon.

We hold foreign crumbs to an invariant. A foreign crumb must satisfy one of three conditions: (a) it is formally proven to satisfy the relevant memory and ABI contract; (b) it executes behind an isolation boundary that prevents violation of Omega’s memory model; or (c) it explicitly weakens the enclosing crumb’s guarantee, and that weakening is visible in the specification and the receipt. For ordinary foreign components our preference is isolation plus contracts. Anything looser recreates `unsafe` in another file. A behavioral receipt alone cannot preserve memory safety for foreign code running in the same address space: without a proof or a boundary, the receipt describes behavior we cannot enforce, and a crumb built on it makes a guarantee it does not hold.

By “isolation boundary” we mean something concrete: the foreign code cannot read or write Omega-managed memory except through the checked ABI, and it shares no mutable address space with Omega code. The canonical mechanism is a hardened OS process. Omega already executes native candidates this way: the runtime places the code on execute-only pages, forks a child, runs the candidate there, and kills it on timeout (`src/runtime/rx_omega.c`). That is crash and hang

containment, not yet a security sandbox: the current child has no seccomp filter, no namespace or filesystem isolation, no resource limits, and no privilege dropping. The hardened form closes those gaps. The child starts with no ambient file descriptors, no network, a seccomp allowlist, namespace and filesystem isolation where the platform provides it, explicit memory and CPU limits, no-new-privileges, and a killable timeout. We keep the separate-process model rather than WASM because Omega must test the actual native realization it may promote; validating WASM is not validating the AArch64 code that will run. WASM remains a separate execution class for portable tools. The architectural rule is: foreign code is data until a disposable, capability-empty native worker verifies it; only receipt-bound verified code may cross into the resident execution path.

One caution about the mechanism itself. A forked child inherits the readable contents of the parent's memory; closing descriptors and adding syscall restrictions does not erase that information. Worker initialization must therefore start from a clean, minimal state: the child is given only what the task requires, and the rest of the parent's address space is cleared or dropped before the foreign code runs.

A promotion rule follows from the invariant. Foreign code qualified through isolation remains isolated in production: if the receipt that admits a component rests on an isolation boundary, the component stays behind that boundary in every deployment. Resident-address-space execution requires the relevant memory/ABI proof, or an explicitly propagated weakening of the enclosing guarantee under condition (c) above. We do not let a component ride an isolation receipt into the resident address space; that would be **unsafe** wearing a sealed label.

A weakening under (c) is not a note in prose. It is a requirement on CLAIM_ID and spec composition that the weakened guarantee is part of the specification itself: SPEC_ID is computed over the weakened specification, so every dependent CLAIM_ID is derived from the weakened claim, and the admission check rejects any seal whose dependency closure treats the component as unweakened. The weakening propagates mechanically through the identifier chain and the admission checks, not by convention.

This preserves the doctrine better than embedding an unverified language inside the verified one. There is no region of Omega where verification stops applying, because the unverified work never enters Omega at all. It stands at the boundary, labeled, receipted, and accountable.

9 Bootstrapping path

The foundation crumbs, the verified core the language is built from, must be verified the hard way first: written carefully, checked thoroughly, with no existing Crumblin to stand on. This is slow. There is no way to skip it, because a crumb store is only as trustworthy as its first entries.

The expected shape is therefore one we accept: a slow, careful beginning, then compounding returns. Each verified foundation crumb makes the next layer cheaper to verify, because new crumbs are built from old verified ones. The library grows slowly at first and then quickly, and every entry pays for itself across all future work. We accept patience at the start as part of the design.

10 Objections

We have stated our proposal. We now consider the objections we expect to be raised against it, and answer each in turn.

Objection 1: This is only Rust with nicer syntax.

We answer that this description would be a smaller idea, and it misses the proposal. A nicer Rust would still manipulate text and hope meaning follows. Our distinguishing claim is different in kind: verification as a four-stage pipeline (**check**, **build**, **verify**, **seal**), the crumb as the unit of reuse, and meaning rather than text as the object the language manipulates. We are not improving Rust's answers. We are asking a different question.

Objection 2: Or Go with a borrow checker.

The same answer applies. Go's simplicity is worth borrowing, and we borrow it openly. But a Go with stricter checking would still be a language about text, with verification as an afterthought. We make verification and sealing structural requirements of reusable composition, and we make the module system the store of verified meanings. The resemblance to Go is in the feel of writing concurrent code, not in the nature of the language.

Objection 3: Most tasks do not need any of this.

We agree. Omega is not a general-purpose scripting language. Where a task does not need safety guarantees or verification, we recommend the simpler tool. A language that insists on proving everything is the wrong instrument for work that needs proving nothing. We do not propose Omega for all programming. We propose it for the programming that has to be right.

Objection 4: A language about text would be simpler and just as good.

It would be simpler to build, and it would not be as good. Text is packaging: two surface texts that elaborate into the identical normalized semantic graph share one identity, and a language that cannot see this will verify the same meaning twice, store ten spellings of one idea, and mistake reformatting for change. We do not claim to decide equivalence of arbitrary programs; where the semantic graphs differ, equivalence is a separate proven artifact. We accept the cost of canonicalizing meaning because the alternative is a system that confuses the wrapper for the thing wrapped.

Objection 5: This is academic theorizing, not engineering.

We measure Omega's success in systems shipped, not papers published. It is a working language for building real systems, or it is nothing. The proposal is written down precisely so that it can be tested against reality, and we will discard whatever reality rejects.

Objection 6: Why should anyone believe these claims before the language exists?

They should not. This document is a design proposal dated October 2026, and every claim in it is provisional until we have built, measured, and ratified it. We propose; we do not assert. The objections above are welcome, and we expect more. A proposal that cannot survive its critics does not deserve to be built.

Objection 7: This has all been done before.

Much of it has, and we say so plainly. Unison addressed code by the hash of its content. Dafny, F*, SPARK, and Lean made verification a first-class concern of the language rather than an afterthought. Proof-carrying code, introduced by Necula in 1997, is the direct ancestor of our maxim that the receipt travels with the code. We do not claim to be the first to put verification inside a programming language.

What we propose is the integration of four things these systems do not combine: semantic identity as the unit of reuse, mandatory evidence-bearing dependencies, an admission policy that decides what may be built upon, and a provenance record of everything the machine has built and verified. Each of those has precedent. Their combination, made structural rather than optional, is the proposal.